COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 DATA STRUCTURES AND ALGORITHMS	**SECTION:**	CPE21S3
DATE PERFORMED:	OCTOBER 15 2026	**DATE SUBMITTED:**	OCTOBER 19 2026
NAME:	JARED ISAIAH DELA CRUZ DUSTIN CARL MAPAS	**INSTRUCTOR:**	ENGR. JIMLORD QUEJADO

ACTIVITY NO. 13.1 **Hands-on Activity 13.1 Parallel Algorithms and Multithreading**

1. PROCEDURE OUTPUT

Screenshot

Code:

```cpp
#include <iostream>
#include <vector>
#include <thread>

void print(int n, const std::string &str) {
    std::cout << "Printing integer: " << n << std::endl;
    std::cout << "Printing string: " << str << std::endl;
}

int main() {
    std::thread t1(print, 10, "T.I.P.");
    t1.join();

return 0;
}
```

Outputs:

DELA CRUZ

```
Printing integer: 10
Printing string: T.I.P.

[Done] exited with code=0 in 0.512 seconds
```

MAPAS

```
Printing integer: 10
Printing string: T.I.P.

[Done] exited with code=0 in 0.504 seconds
```

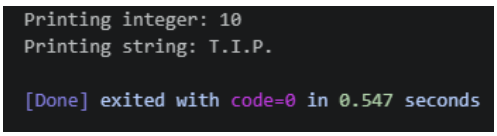
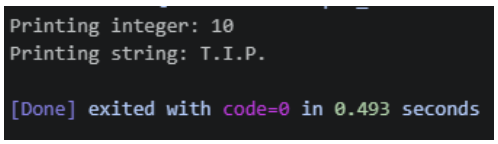
	 
Analysis	In this simple one threaded task, this program uses thread and defines a print function to print and integer and string. Moreover, it creates a separate thread (t1) to run this function in the background using the values 10, and “T.I.P. Lastly, the t1.join() is used to ensure that the main program will wait until the thread t1 is finished. This is done to avoid the main thread and t1 to concurrently run.

Table 13-1. Simple One-Threaded Example

Screenshot	Code:

```cpp
#include <iostream>
#include <vector>
#include <thread>

void print(int n, const std::string &str) {
    std::string msg = std::to_string(n) + " : " + str;
    std::cout << msg << std::endl;
}

int main() {
    std::vector<std::string> s = {
    "T.I.P.",
    "Competent",
    "Computer",
    "Engineers"
};

std::vector<std::thread> threads;
for (int i = 0; i < s.size(); i++) {
    threads.push_back(std::thread(print, i, s[i]));
}

for (auto &th : threads) {
    th.join();
}

return 0;
}
```

Outputs:

Dela Cruz

```
3 : Engineers
1 : Competent
0 : T.I.P.
2 : Computer

[Done] exited with code=0 in 0.537 seconds
```

```
0 : T.I.P.
1 : Competent
2 : Computer
3 : Engineers

[Done] exited with code=0 in 0.541 seconds
```

Mapas

```
0 : T.I.P.
1 : Competent
3 : Engineers
2 : Computer

[Done] exited with code=0 in 0.558 seconds
```

```
0 : T.I.P.1 : Competent

3 : Engineers
2 : Computer

[Done] exited with code=0 in 0.536 seconds
```

Analysis	This task presents a multithread example, it creates a separate thread for each string in the list, passing its index, and the string to be printed. Furthermore, because all four threads run at the exact same time , the output will be randomized or different in each run because those threads are concurrently running. Lastly, it loops through every thread so that every thread finishes before the program ends.

Table 13-2. Multithreaded Example.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Part A: Demonstrate an understanding of parallelism, concurrency, and multithreading in C++ by answering the given questions. Use of supplementary materials to support answers must be cited as reference.

QUESTION: Write a definition of multithreading and its advantages/disadvantages.

ANSWERS:
Dela Cruz - Multithreading is a programming method where a single program is split into smaller parts called threads that can run at the same time. Its main advantage is that it makes applications faster and keeps your screen from freezing during heavy tasks, but its major disadvantage is complexity, as shared memory can cause tricky bugs and data errors.

Mapas - Multithreading is a technique wherein a program is split into smaller independent tasks called threads that can run at the same time. One of its advantages is that it keeps the program fast and responsive. Although one of its disadvantages is that is the increase in the complexity of code which can be hard to debug when needed.

QUESTION: Rationalize the use of multithreading by providing at least 3 use-cases.

ANSWERS:
Dela Cruz - Multithreading is used in web browsers so that each open tab runs independently and a crash on one site doesn't freeze the others. Video games also use it to manage user inputs, character physics, and graphics rendering simultaneously for smooth gameplay, while download managers use it to grab different parts of a large file all at once to finish faster.

Mapas - Multithreading can be used in video games by handling user inputs, physics, and simultaneous rendering. It can also be used in web servers to handle many visitors requests at the same time without slowing down. And lastly, video editing software also uses multithreading to render heavy effects in the background.

QUESTION: Differentiate between parallelism and concurrency.

ANSWERS:
Dela Cruz - Concurrency is about structuring a program to handle multiple tasks by quickly switching back and forth between them on a single processor core, making it look like they happen together. Parallelism, however, is about actual physical execution where multiple tasks truly happen at the exact same instant using separate hardware cores.

Mapas - Parallelism means executing multiple tasks at the exact same time using separate CPU cores. While concurrency is for managing multiple tasks by switching rapidly between them on a single core.

Part B

Code

```cpp
#include <iostream>
#include <thread>

int global_var = 0;

void add(int value) {
    global_var += value;
}

int main() {
    //Createing 3 threads
    std::thread T1([>>]add, 10);
    std::thread T2([>>]add, 20);
    std::thread T3([>>]add, 30);

    std::cout << "Display (Initial before T1 join): " << global_var << std::endl;
    T1.join();

    std::cout << "Display (After T1 join): " << global_var << std::endl;
    T2.join();

    std::cout << "Display (After T2 join): " << global_var << std::endl;
    T3.join();

    std::cout << "Display (Final/After T3 join): " << global_var << std::endl;

    return 0;
}
```

Output	`Display (Initial before T1 join): 0`
	`Display (After T1 join): 60`
	`Display (After T2 join): 60`
	`Display (Final/After T3 join): 60`

Analysis

Dela Cruz	For this program, three separate threads were created to add values to a shared global variable. Because all three threads executed so quickly in the background, they had already finished updating the variable before the program waited for the first thread to complete. As a result, the total jumped straight from 0 to 60 right after the first join() call, while the remaining join() calls simply displayed the same final total.
Mapas	For this activity output, all three threads were executed quickly in the background so that by the time T1.join() finished T2 and T3 had already completed. This caused glovalvar to jump directly from 0 to 60right after the first join. Hence the following join() calls just displayed the final total without any changes.

Part C

Code

```cpp
#include <iostream>
#include <thread>
#include <utility>

template <typename T>
void bubbleSort(T arr[], size_t arrSize) {
    for (size_t i = 0; i < arrSize; i++) {
        for (size_t j = i + 1; j < arrSize; j++) {
            if (arr[i] > arr[j]) {
                std::swap(&arr[i], &arr[j]);
            }
        }
    }
}

void printArray(const std::string& label, const int arr[], size_t size) {
    std::cout << label << ": ";
    for (size_t i = 0; i < size; i++) {
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;
}
```

```cpp
int main() {
    int arr1[] = {64, 34, 25, 12, 22, 11, 90};
    int arr2[] = {88, 14, 45, 6, 71, 30, 52};

    size_t size1 = sizeof(arr1) / sizeof(arr1[0]);
    size_t size2 = sizeof(arr2) / sizeof(arr2[0]);

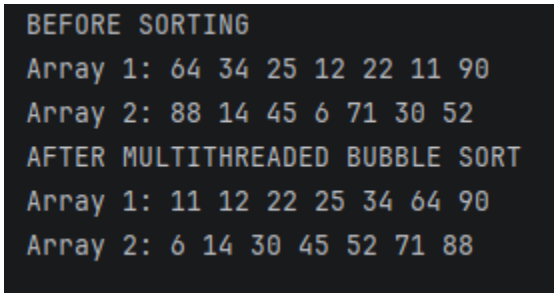
    std::cout << "BEFORE SORTING" << std::endl;
    printArray( label: "Array 1", arr1, size1);
    printArray( label: "Array 2", arr2, size2);

    //multithreading forlaunching 2 threads to sort independent arrays concurrently
    std::thread t1(>>bubbleSort<int>, >>arr1, >>size1);
    std::thread t2(>>bubbleSort<int>, >>arr2, >>size2);

    t1.join();
    t2.join();

    std::cout << "AFTER MULTITHREADED BUBBLE SORT" << std::endl;
    printArray( label: "Array 1", arr1, size1);
    printArray( label: "Array 2", arr2, size2);

    return 0;
}
```

Output	

Analysis

Dela Cruz	Now for this program, two separate threads were launched to run the bubble sort algorithm on two different arrays at the exact same time. Because the arrays were independent, both threads were able to sort their respective data concurrently without affecting each other. Using the join() function ensured that the main program waited for both sorting operations to finish before printing the final, correctly ordered arrays.
Mapas	In the output it shows that by running two separate threads, both arrays were sorted concurrently without interfering with one another. Moreover the join() ensures the main program will wait for the tasks to be finished before printing hence resulting in a successfully sorted array.

Conclusions

Dela Cruz	Through this activity, I gained a clearer and better understanding of multithreading, concurrency, and parallelism in C++ code language. I was able to observe how splitting tasks across multiple threads allows programs to process data faster and handle operations concurrently. Overall, this lab helped me appreciate the benefits and practical applications of parallel algorithms in improving software performance
Mapas	After doing this activity I learned about multi-threading, and how it works. I also created a c++ code that uses parallel algorithms for better performance. Moreover, I also created a code that implements multithreading and concurrency. I also learned about concurrency and parallelism. Furthermore, I also did analysis of the procedure and the supplementary part. Lastly, I still have a lot to learn more and I will keep learning to better understand this topic.

REFERENCES:

Microsoft. (2025). About processes and threads. Microsoft Learn.
https://learn.microsoft.com/en-us/windows/win32/procthread/about-processes-and-threads

Oracle. (n.d.). Class Thread. Oracle Help Center.
https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/Thread.html

GeeksforGeeks. (2026, August 31). *Multithreading in C++*.
https://www.geeksforgeeks.org/cpp/multithreading-in-cpp/

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explanation of concepts**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **We used Gemini as a tool for understanding**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.